First Negative Integer in Every Window of Size k

A C program using pointers and a nested-loop sliding window approach

Name : Rudra Pratap Singh

SAP ID : 590032811

Batch : 16

1. Problem Statement

Write a C program that reads the size and the elements of a variable-length integer array arr[] and an integer k such that $0 < k \leq$ size. The array must be passed to a function named first_negative() using its address, and pointers must be used to pass the array between the calling function and first_negative().

The function first_negative() must:

- Consider every contiguous subarray of size k as a sliding window.
- Find the first negative integer in each window while moving from left to right.
- Display the first negative integer in the order of the corresponding windows.
- Display 0 if a window does not contain any negative integer.
- Separate the results by spaces.
- Display -1 if k is invalid.
- Use a nested-loop approach to solve the problem.

2. Approach and Algorithm

An array of size n has exactly $n - k + 1$ windows of size k. The window that starts at index i covers the elements arr[i] to arr[i + k − 1]. The program uses two loops:

- Outer loop (i): selects the starting index of each window, from 0 to size − k.
- Inner loop (j): scans the current window from left to right and stops at the first negative element.

Algorithm

1. Start.
2. Read size and allocate memory for size integers dynamically using malloc().
3. Read the elements of the array and then read k.
4. Call first_negative(arr, size, k), passing the base address of the array.
5. If $k \leq 0$ or $k >$ size, print -1 and return.
6. For $i = 0$ to size − k, set found = 0 and repeat steps 7 and 8.
7. For $j = i$ to $i + k - 1$: if *(arr + j) < 0, print it, set found = 1 and break out of the inner loop.
8. If found is still 0, print 0 for this window.
9. Free the allocated memory and stop.

Time and Space Complexity

For each of the (n − k + 1) windows the inner loop runs at most k times, so the time complexity is O(n × k). Only a few extra variables are used apart from the input array, so the extra space required is O(1).

3. Key Concepts Used

- Dynamic memory allocation: malloc() creates an array whose size is decided at run time, and free() releases it.
- Pointers: the array is passed as int *arr, and elements are accessed with pointer notation *(arr + j).
- Nested loops: the outer loop moves the window and the inner loop searches inside it.
- Input validation: an invalid k (k ≤ 0 or k > size) prints -1.

4. Source Code

```
#include <stdio.h>
#include <stdlib.h>

/* Prints the first negative integer of every window of size k.
   The array is received through a pointer (its address). */
void first_negative(int *arr, int size, int k)
{
    int i, j, found;

    /* Invalid window size */
    if (k <= 0 || k > size)
    {
        printf("-1\n");
        return;
    }

    /* Outer loop: starting index of each window */
    for (i = 0; i <= size - k; i++)
    {
        found = 0;

        /* Inner loop: scan the current window left to right */
        for (j = i; j < i + k; j++)
        {
            if (*(arr + j) < 0)
            {
                printf("%d ", *(arr + j));
                found = 1;
                break;              /* first negative found */
            }
        }

        if (!found)
            printf("0 ");           /* no negative in this window */
    }
    printf("\n");
```

```
}

int main(void)
{
    int size, k, i;
    int *arr;

    printf("Enter the size of the array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("-1\n");
        return 0;
    }

    arr = (int *)malloc(size * sizeof(int));
    if (arr == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Enter %d elements: ", size);
    for (i = 0; i < size; i++)
        scanf("%d", arr + i);

    printf("Enter the window size k: ");
    scanf("%d", &k);

    printf("Output: ");
    first_negative(arr, size, k);   /* passing address of array */

    free(arr);
    return 0;
}
```

5. Sample Input and Output

The program was compiled with gcc using the flags -Wall -Wextra (no warnings) and tested with all four examples given in the problem statement.

Test Case 1

```
Enter the size of the array: 8
Enter 8 elements: 12 -1 -7 8 -15 30 16 28
Enter the window size k: 3
Output: -1 -1 -7 -15 -15 0
```

Test Case 2

```
Enter the size of the array: 6
Enter 6 elements: 1 2 -3 4 5 -6
Enter the window size k: 2
```

```
Output: 0 -3 -3 0 -6
```

Test Case 3

```
Enter the size of the array: 4
Enter 4 elements: 1 2 3 4
Enter the window size k: 2
Output: 0 0 0
```

Test Case 4

```
Enter the size of the array: 3
Enter 3 elements: 1 2 3
Enter the window size k: 4
Output: -1
```

6. Test Result Summary

No.	Array	k	Expected Output	Result
1	12 -1 -7 8 -15 30 16 28	3	-1 -1 -7 -15 -15 0	Pass
2	1 2 -3 4 5 -6	2	0 -3 -3 0 -6	Pass
3	1 2 3 4	2	0 0 0	Pass
4	1 2 3	4	-1	Pass

All outputs match the expected results.

7. Explanation of Test Case 1

For the array 12 -1 -7 8 -15 30 16 28 with k = 3, there are 6 windows:

- [12, -1, -7] → first negative is -1
- [-1, -7, 8] → first negative is -1
- [-7, 8, -15] → first negative is -7
- [8, -15, 30] → first negative is -15
- [-15, 30, 16] → first negative is -15
- [30, 16, 28] → no negative number, so 0 is displayed

Hence the output is: -1 -1 -7 -15 -15 0.

8. Conclusion

The program correctly finds the first negative integer in every window of size k using a nested-loop approach. The array is created dynamically, passed to first_negative() by its address and accessed entirely through pointers. Invalid values of k are handled by printing -1, and the program produced the expected result for every sample case.